

Sécurité du système de fichiers

Stéphane Gill

Stephane.Gill@CollegeAhuntsic.qc.ca

Table des matières

Introduction.....	2
Gestion des droits d'accès.....	2
Les droits d'accès hors répertoire.....	2
Les fichiers de périphériques.....	2
Droits d'accès étendus.....	3
Attributs de fichiers.....	4
ACL Posix.....	4
Prise en compte de ACL's.....	5
Utilisation.....	5
ACL par défaut.....	7
Persistance des ACLs lors des manipulations de fichiers.....	8
Références.....	9

Document écrit par Stéphane Gill

© Copyright 2013 Stéphane Gill

Ce document est soumis à la licence GNU FDL. Permission vous est donnée de distribuer et/ou modifier des copies de ce document tant que cette note apparaît clairement.

Introduction

La sécurité des systèmes de fichiers est généralement fondée sur les permissions, ce qui signifie que l'accès aux différentes ressources du système est déterminé explicitement par des droits d'accès et propriété. Ce modèle de sécurité du système de fichiers fonctionne généralement assez bien. Toutefois, il ne pourra pas opérer de manière satisfaisante si les étapes appropriées à son utilisation ne sont pas suivies.

Dans ce document, les risques liés à la gestion des droits d'accès sont d'abord présenté. Ensuite, l'utilisation des ACL Posix est exploré.

Gestion des droits d'accès

Les droits d'accès hors répertoire

En plus des droits d'accès habituels, il existe deux droits d'accès spéciaux pour les répertoires. Le premier droits d'accès est le « sticky bit ». Lorsqu'il est appliqué aux droits d'un répertoire, le sticky bit crée une situation particulière. Ainsi, le « Sticky bit » fait en sorte que seul le propriétaire d'un fichier peut l'effacer, et ce, même si tous les utilisateurs ont des droits de lecture et d'écriture. La valeur octale de ce droit est 1000 et elle peut être modifiée par la commande *chmod*.

Le second droits d'accès est le bit SGID au répertoire. Lorsque ce droit d'accès est appliqué à un répertoire, tout fichier créé à l'intérieur du répertoire est créé avec le même groupe propriétaire.

Les fichiers de périphériques

Il existe certains fichiers qui se réfèrent aux périphériques du système, il résident pour la plupart dans le répertoire */dev*. Heureusement, dans la majorité des distributions les droits d'accès de ces fichiers sont configuré correctement. Toutefois, il existe deux situations où les droits d'accès des fichiers périphériques peuvent posé problème :

- Un fichier de périphérique de disque possède des droits d'accès qui permettent aux utilisateurs d'accéder directement aux périphériques. Pour corriger la situation il faut utiliser la commande `chmod 600`.
- Un périphérique quelconque possédant des permissions publiques. Pour corriger la situation, il faut utiliser la commande `chmod o-rwx`.

Droits d'accès étendus

Le plus grand facteur de complication du système de droit d'accès de Linux est probablement les droits d'accès étendus. Ces permettent à un utilisateur d'assouplir les contraintes des droits d'accès. Nous allons utiliser l'exemple de la commande *passwd* pour illustrer ce concept. Observons les droits d'accès du fichier */etc/passwd*.

```
# ls -l /etc/passwd
-rw-r--r-- 1 root root 1325 Sep 5 07:48 /etc/passwd
```

La commande *passwd* permet aux utilisateurs ordinaires de modifier leurs mots de passe même s'ils n'ont pas de droit en écriture sur le fichier */etc/passwd*. Linux utilise le concept des droits d'accès étendus afin de permettre aux utilisateurs de modifier leur mot de passe. Ce concept permet, le temps du traitement, d'affecter à un programme le numéro d'utilisateur du propriétaire du fichier ainsi que de son groupe. Le premier des deux mécanismes est autorisé par le SUIDbit (Set User ID bit, valeur octale 2000), le second par le SGIDbit (Set Group ID bit, valeur octale 4000).

Il est important de vérifier la présence de nouveaux exécutables avec des droits étendus. La commande suivante permet de faire des fichiers ayant le SUID/SGID activé.

```
find / -perm +6000 -follow > /tmp/suid_sgid.list
```

Ensuite, il est possible d'écrire un script qui permettra de vérifier la présence de nouveaux binaires SUID/SGID.

```
for BINFILE in $(find / -perm +6000 -follow) ; do
    if [ «$(grep $BINFILE /tmp/suid_sgid.list)» = «»]; then
        Echo $BINFILE | mail -s «New SUID/SGID!» root
    fi
done
```

Attributs de fichiers

Le système de fichiers de Linux offre des paramètres additionnels. Ces propriétés sont appelées attributs de fichiers et elles sont contrôlées par la commande *chattr*. La commande *chattr* s'utilise comme suit :

```
chattr [+|-|=] modes fichier [fichier...]
```

Où modes représente un des deux modes suivants :

- Le mode a (*append*, ajout) empêche tout accès sans ajout à la suite du fichier. Ainsi par exemple, seul les ajouts sont autorisés dans les fichiers de journaux.
- Le mode i (*immutable*, inaltérable) empêche toute modification, quelle qu'elle soit.

Ces deux modes sont mutuellement exclusifs. Ils ont préséance sur les droits d'accès ordinaires et ne peuvent être activés ou désactivés que par *root*.

La commande *lsattr* est une version spéciale de la commande *ls*, qui affiche les attributs à la place des droits d'accès.

ACL Posix

La mise en oeuvre de droits sur les fichiers d'un système Linux peut parfois manquer de souplesse. En effet, un fichier peut se voir affecter des droits pour trois types d'utilisateur différents : le propriétaire, le groupe propriétaire et « le reste du monde ».

Imaginons un instant un fichier *exemple.txt*. L'utilisateur *stephane* (propriétaire du fichier) peut le lire et le modifier. Les utilisateurs du groupe propriétaire peuvent juste le consulter. Les autres utilisateurs n'ont aucun droit.

On souhaite donner des droits en écriture sur le fichier *exemple.txt* à l'utilisateur *nathalie*. L'utilisateur *nathalie* n'est ni propriétaire et n'est ni membre du groupe propriétaire du fichier.

De fait, l'absence de gestion de droits atomiques (droits utilisés sur les fichiers Windows de type NTFS) ne permet pas à un système Linux d'attribuer ce type de droit.

Depuis quelque années, il est possible d'attribuer de tels droits grâce à l'utilisation des ACL (Listes de Contrôle d'Accès). Disponible avec la majorité des distributions de Linux ou sur le site

<http://acl.bestbits.at>, la mise en oeuvre de ces listes est conforme à la norme POSIX (Portable Operating System Interface).

Prise en compte de ACL's

Sous CentOS 6, il n'est pas nécessaire de recompiler le noyau pour que les ACL soient pris en compte. Pour activer la prise en charge des ACL par le système de fichiers, et ce, pour chaque partition souhaitée, il faudra exécuter deux actions distinctes. Dans l'exemple qui suit, on ne souhaite activer les ACL que sur `/dev/hda1`.

L'activation des ACL sur la partition `/dev/hda1` s'effectue à l'aide de la commande suivante :

```
# mount -o remount,acl /dev/hda1
```

Pour automatiser l'activation des ACL au démarrage, il faut modifier le fichier `/etc/fstab` en ajoutant ACL aux options à transmettre au montage. Par exemple la ligne :

```
/dev/hda1 / ext3 errors=remount-ro 0 1
```

deviendrait :

```
/dev/hda1 / ext3 acl,errors=remount-ro 0 1
```

Utilisation

Avant de décrire les commandes qui permettent de modifier les ACL voici trois définitions utiles :

- **ACL minimale** : Composée exclusivement d'éléments de type propriétaire, groupe et « reste du monde », l'ACL minimale est une traduction « en ACL » des droits d'accès traditionnels Unix.
- **ACL étendue** : L'ACL étendue prolonge les droits de l'ACL minimale. Elle contient au moins un élément de type mask et peut contenir des éléments de type utilisateur et/ou groupe.
- **ACL par défaut** : Les ACL par défaut ne peuvent être appliquées qu'aux répertoires et définissent de quels droits un objet du système de fichiers devra hériter (de son répertoire parent) lors de sa création.

Affichage des ACL

La commande *getfacl* permet d'afficher les ACL associées à un fichier ou un répertoire. La syntaxe de la commande est la suivante :

```
$ getfacl nom_fichier
```

Modification des ACL

La modification des ACL d'un fichier, s'effectue en utilisant la commande *setfacl* avec l'option -m (modify). Seul le propriétaire du fichier ou l'administrateur (root) est en mesure d'effectuer cette modification. .

La modification des droits d'un utilisateur, s'effectue en utilisant le paramètre u, suivi du nom ou de l'uid [2] de l'utilisateur, suivi des droits à affecter en respectant le format u[ser] [:]uid [:perms]. Si l'uid est vide, le propriétaire sera utilisé.

Exemple :

Pour permettre à l'utilisateur cecile d'écrire dans un fichier appartenant à l'utilisateur loick (fichier pour lequel elle n'a normalement que des droits en lecture seule) on utilisera la commande :

```
$ setfacl -m u:cecile:w test.acl
```

Pour modifier les droits d'un groupe, on utilisera le paramètre g, suivi du nom ou du gid [3] du groupe, suivi des droits à affecter en respectant le format g[roup] [:]gid [:perms].

Exemple :

Pour permettre au groupe admin d'écrire dans un fichier appartenant à l'utilisateur loick (fichier pour lequel les membres du groupe admin n'ont normalement que des droits en lecture seule) on utilisera la commande :

```
$ setfacl -m g:admin:w test.acl
```

Pour modifier les droits du reste du monde (other), on utilisera le paramètre o, suivi des droits à affecter en respectant le format o[ther][:] [:perms]

Suppression d'une ACL

Lorsqu'on souhaite supprimer une ACL, on ne peut, en fait, que supprimer les éléments de type mask et/ou utilisateur/groupe. Le maximum des entrées de l'ACL supprimée, il restera quand

même (et c'est on ne peut plus logique) les entrées correspondantes aux droits unix du fichier concerné.

Exemple 1 :

Pour détruire toutes les entrées d'une ACL étendue :

```
$ setfacl -b mon.fichier
```

Exemple 2 :

Dans l'exemple ci-dessus, on enlève de l'ACL toutes les entrées correspondantes au groupe users.

```
$ setfacl -x g:users mon.fichier
```

ACL par défaut

Une ACL par défaut s'applique à un répertoire. Ce type d'ACL permet de définir un certain nombre de paramètres qui seront appliqués automatiquement et par défaut à tout fichier ou répertoire créé sous le répertoire de départ. L'intérêt consiste en la mise en oeuvre de procédures globales de gestion des droits ACL au sein d'une arborescence donnée.

Création d'une ACL par défaut

La commande *setfacl* utilisé avec l'option *-m* et du paramètre *d* permet d'attribuer des ACL par défaut à un répertoire.

Exemple :

Soit le répertoire *mon.repertoire* (*drwxr-xr-x loick users*). On décide qu'à partir de maintenant, cecile pourra lire et écrire dans les fichiers/répertoires créés sous *mon.repertoire*. On crée donc l'ACL par défaut :

```
$ setfacl -m d:u:cecile:rw mon.repertoire
```

On vérifie :

```
$ getfacl mon.repertoire
# file: mon.repertoire
```

```
# owner: loick
# group: users
user::rwx
group::r-x
other::r-x
default:user::rwx
default:user:cecile:rw-
default:group::r-x
default:mask::rwx
default:other::r-x
```

Suppression d'une ACL par défaut

La suppression d'un ACL par défaut s'effectue à l'aide de la commande *setfacl* avec l'option *-k*. La syntaxe est la suivante :

```
$ setfacl -k mon.repertoire
```

Persistence des ACLs lors des manipulations de fichiers

Malgré l'intégration des ACL par le système, leur gestion peut être mal ou pas prise en compte par certaines applications. On distingue deux groupes d'applications différents.

Applications de sauvegarde.

Les applications standard de sauvegarde (tar, cpio...), à l'exception de *star*, ne savent pas garantir la prise en compte complète des ACL. *star* est disponible sur le site Web <http://freshmeat.net/projects/star>.

Il y a aussi des logiciels comme *Arkeia* qui propose une solution de sauvegarde complète avec le support des ACL.

Applications de manipulations de fichier

Si les paquetages sont correctement mis à jour (coreutils entre autres), les commandes classiques (cp, mv, ls) prennent en compte les ACL. Pour les éditeurs et les gestionnaires de fichiers moins conventionnels, cela dépend du principe de fonctionnement de l'application. Si les modifications sont apportées dans le fichier d'origine, l'ACL est conservée, dans le cas contraire il y a toutes les chances que les ACL soient perdues...

Références

Aron Hsiao, « Sécurité sous Linux », Campus press, 2001.

« Red Hat Linux 9 : Red Hat Linux Security Guide », Red Hat Inc., 2002.